

- TomatoMall 项目面试准备文档
 -  目录
 - 1. Redis缓存体系
 - 1.1 项目中Redis使用概览
 - 1.2 缓存三大问题（八股文+项目实施）
 -  问题一：缓存穿透（Cache Penetration）
 - 什么是缓存穿透？
 - 解决方案对比
 -  项目实施：缓存空值
 -  进阶：布隆过滤器（八股文）
 -  问题二：缓存击穿（Cache Breakdown）
 - 什么是缓存击穿？
 - 解决方案对比
 -  项目实施：分布式锁 + 双重检查
 -  进阶：逻辑过期（八股文）
 -  问题三：缓存雪崩（Cache Avalanche）
 - 什么是缓存雪崩？
 - 解决方案对比
 -  项目实施：TTL随机化 + 缓存预热
 -  进阶：多级缓存（八股文）
 - 1.3 缓存更新策略（Cache Aside Pattern）
 - 什么是Cache Aside（旁路缓存）？
 - 项目实施
 - 为什么先更新DB，再删除缓存？
 - 面试话术
 - 2. 库存防超卖机制
 - 2.1 什么是超卖？
 - 2.2 传统方案的问题
 - 2.3 Redis原子操作方案
 - 原理
 - 项目实施
 - 性能对比
 - 2.4 完整下单流程
 - 面试话术
 - 3. 分布式一致性方案
 - 3.1 一致性问题分析
 - 3.2 项目实施

- 支付成功同步MySQL
 - 订单超时回滚
 - 延迟队列定时任务
- 3.3 延迟队列实现（ZSet）
- 3.4 数据一致性保障
- 面试话术
- 4. JWT + Spring Security认证方案
 - 4.1 当前实现分析
 - 架构
 - JWT生成
 - 拦截器验证
 - 4.2 标准实现方案
 - 架构设计
 - JWT工具类（优化版）
 - JWT认证过滤器（核心）
 - Spring Security配置
 - 异常处理
 - 获取当前用户
 - 4.3 对比分析
 - 面试话术
- 5. 面试高频问题汇总
 - 5.1 分布式事务与库存一致性
 - 5.2 高并发下的库存超卖问题
 - 5.3 订单超时处理的可靠性
 - 5.4 JWT的安全问题
 - 5.5 数据库设计与索引优化
 - 5.6 支付回调的幂等性
 - 5.7 项目难点与收获
 - 5.8 系统设计题
-  面试准备清单
 - 重点准备的主题
 - 自我检查清单
-  总结
 - 项目亮点
 - 面试策略



目录

- 1. Redis缓存体系
- 2. 库存防超卖机制
- 3. 分布式一致性方案
- 4. JWT+Spring Security认证
- 5. 面试高频问题汇总

1. Redis缓存体系

1.1 项目中Redis使用概览

Redis 功能模块		
基础功能(4个)	进阶功能(3个)	高级功能(3个)
• 用户会话 (String) • 商品详情缓存 (Hash) • 商品收藏 (Set) • 首页广告缓存 (List)	• 库存防超卖 (String原子) • 购物车 (Hash) • 商品热搜榜 (ZSet)	• 缓存三大问题解决方案 (穿透/击穿/雪崩) • 超时订单延迟队列 (ZSet) • PV/UV 统计 (String + HyperLogLog)

1.2 缓存三大问题（八股文+项目实现）



问题一：缓存穿透（Cache Penetration）

什么是缓存穿透？

定义：查询一个不存在的数据，缓存中没有，每次都打到数据库

场景：

- 1. 恶意攻击：大量请求查询不存在的商品ID（如ID=99999）
- 2. 业务逻辑：第一次查询新商品，数据库确实不存在

问题：

- 缓存无法命中（不存在的数据不会缓存）
- 每次请求都穿透缓存打到数据库
- 数据库压力过大，可能被打挂

解决方案对比

方案	原理	优点	缺点	适用场景
缓存空值	查询为null时，缓存特殊标记	简单易实现	占用内存，可能缓存大量无效数据	数据命中率高，恶意攻击少
布隆过滤器	使用位图快速判断数据是否存在	内存占用极小，查询O(k)	有误判率，不支持删除	海量数据，命中率高
Redis布隆	Redisson实现布隆过滤器	分布式支持，自动过期	有误判率	分布式场景
互斥锁	第一个请求查DB，其他等待	保证只查一次DB	性能差，阻塞其他请求	并发量小的场景

✅ 项目实施：缓存空值

核心代码：[ProductCacheServiceImpl.java:235-240](#)

```
/**
 * 缓存空值
 * 【缓存穿透防护】
 * 当数据库中不存在该数据时，缓存一个空值标记
 * 设置较短的过期时间，避免占用过多内存
 */
private void cacheNullValue(String productId) {
    String cacheKey = PRODUCT_CACHE_PREFIX + productId;
    Map<String, Object> nullMap = new HashMap<>();
    nullMap.put("_null", NULL_CACHE_VALUE); // 特殊标记: NULL_OBJECT
    redisService.hSetAll(cacheKey, nullMap, NULL_CACHE_TTL_MINUTES,
```

```
TimeUnit.MINUTES);
```

```
//
```

↑ 较短TTL：5分钟

```
}
```

读取时的判断：ProductCacheServiceImpl.java:145-148

```
@Override
public ProductVO getProductWithCache(String productId) {
    String cacheKey = PRODUCT_CACHE_PREFIX + productId;

    // 检查缓存是否存在
    if (redisService.hasKey(cacheKey)) {
        Map<Object, Object> cacheMap = redisService.hGetAll(cacheKey);

        // 【关键】检查空值标记（缓存穿透防护）
        if (cacheMap != null && NULL_CACHE_VALUE.equals(cacheMap.get("_null"))) {
            return null; // 数据库中不存在该商品
        }

        if (cacheMap != null && !cacheMap.isEmpty()) {
            return mapToProductVO(cacheMap);
        }
    }
    // ... 后续逻辑
}
```

执行流程：

第一次查询不存在的商品ID=999999：

1. Redis查询 → miss
2. MySQL查询 → null
3. 缓存空值标记（NULL_OBJECT），TTL=5分钟

后续查询ID=999999：

1. Redis查询 → hit（NULL_OBJECT）
2. 直接返回null，不再查MySQL

进阶：布隆过滤器（八股文）

原理：

布隆过滤器 = 位图 + n个哈希函数

添加元素：

1. 对元素值进行n次哈希，得到n个位置
2. 将位图中这n个位置置为1

查询元素：

- 1. 对元素值进行n次哈希
- 2. 检查这n个位置是否都为1
 - 都是1 → 可能存在（有误判）
 - 有任一为0 → 一定不存在

Redis实现：

```
// 使用Redisson实现布隆过滤器
RBloomFilter<String> bloomFilter = redisson.getBloomFilter("product:bloom");
bloomFilter.tryInit(1000000L, 0.01); // 预计100万元素，误判率1%

// 添加商品ID到布隆过滤器
bloomFilter.add(productId);

// 查询商品是否存在
if (!bloomFilter.contains(productId)) {
    return null; // 一定不存在，直接返回
}
// 可能存在，继续查缓存和数据库
```

布隆过滤器 vs 缓存空值：

维度	缓存空值	布隆过滤器
内存占用	每个key占用较多内存	100万数据仅占用1.2MB
准确性	100%准确	有误判率（可配置）
删除操作	直接删除key	不支持删除（需特殊处理）
实现复杂度	简单	需要引入额外组件

面试话术：

对于缓存穿透，我在项目中采用了**缓存空值**方案：

- 当数据库查询为null时，缓存 `NULL_OBJECT` 标记
- 设置较短TTL（5分钟），避免内存浪费

如果数据量达到千万级别，我会考虑使用**布隆过滤器**：

- 预先加载所有有效的商品ID到布隆过滤器
- 查询前先判断，如果布隆过滤器说不存在，直接返回
- 误判率可控制在0.01%，内存占用极小

● 问题二：缓存击穿（Cache Breakdown）

什么是缓存击穿？

定义：某个热点key过期瞬间，大量请求同时打到数据库

场景：

- 热门商品缓存过期（如秒杀商品）
- 热点新闻、微博热搜

时间线：

10:00:00.000 - 商品101缓存过期
10:00:00.001 - 线程A查询 → 缓存miss → 查询DB（100ms）
10:00:00.002 - 线程B查询 → 缓存miss → 查询DB（100ms）
10:00:00.003 - 线程C查询 → 缓存miss → 查询DB（100ms）
...
10:00:00.100 - 1000个线程同时在查询数据库！

结果：

- 数据库CPU飙升
- 请求超时
- 可能导致缓存雪崩

解决方案对比

方案	原理	优点	缺点	适用场景
互斥锁	只让一个线程重建缓存	简单，数据一致性好	其他线程阻塞	热点key，读多写少
逻辑过期	缓存不过期，后台异步更新	无阻塞，性能好	数据不一致（短期）	热点key，允许短暂数据延迟
永不过期	设置TTL为Long.MAX_VALUE	不会过期	需要主动更新	很少变更的热点数据

✅ 项目实现：分布式锁 + 双重检查

核心代码：[ProductCacheServiceImpl.java:155-184](#)

```
@Override
public ProductVO getProductWithCache(String productId) {
    String cacheKey = PRODUCT_CACHE_PREFIX + productId;

    // 1. 【第一次检查】尝试从缓存获取
```

```

if (redisService.hasKey(cacheKey)) {
    Map<Object, Object> cacheMap = redisService.hGetAll(cacheKey);

    // 检查空值标记（缓存穿透防护）
    if (cacheMap != null && NULL_CACHE_VALUE.equals(cacheMap.get("_null"))) {
        return null;
    }

    if (cacheMap != null && !cacheMap.isEmpty()) {
        return mapToProductVO(cacheMap);
    }
}

// 2. 【获取分布式锁】缓存未命中，尝试获取锁重建缓存
String lockKey = CACHE_LOCK_PREFIX + productId;
boolean locked = redisService.setIfAbsent(lockKey, "1", LOCK_TTL_SECONDS,
TimeUnit.SECONDS);
//                                     ↑ SETNX实现，原子操作

try {
    if (locked) {
        // 3. 【获取锁成功】从数据库加载并缓存
        return loadAndCacheProduct(productId);
    } else {
        // 4. 【获取锁失败】短暂等待后重试
        Thread.sleep(50);

        // 5. 【第二次检查 - 双重检查】
        if (redisService.hasKey(cacheKey)) {
            Map<Object, Object> cacheMap = redisService.hGetAll(cacheKey);
            if (cacheMap != null && !cacheMap.isEmpty() &&
                !NULL_CACHE_VALUE.equals(cacheMap.get("_null"))) {
                return mapToProductVO(cacheMap);
            }
        }

        // 6. 【降级】仍然未命中，直接查数据库
        return loadFromDatabase(productId);
    }
} catch (InterruptedException e) {
    Thread.currentThread().interrupt();
    return loadFromDatabase(productId);
} finally {
    // 7. 【释放锁】
    if (locked) {
        redisService.delete(lockKey);
    }
}
}

```

SETNX命令原理：

```

# SETNX = SET if Not eXists
# 如果key不存在，则设置成功返回1

```

```
SETNX lock:product:101 "1"
```

返回1 → 获取锁成功

返回0 → 锁已被占用

热点商品缓存过期时的并发请求:

结果：只有1个线程查DB，999个线程等待50ms后命中缓存

原理：

```
{
  "data": {...},          // 实际数据
  "expireTime": 1234567890 // 逻辑过期时间
}
```

1. 查询缓存时，不设置Redis TTL，而是存储逻辑过期时间
2. 缓存"永不过期"，但通过逻辑过期时间判断是否需要更新
3. 如果逻辑过期，返回旧数据，同时异步更新缓存

```
public ProductVO getProductWithLogicalExpire(String productId) {
    String cacheKey = PRODUCT_CACHE_PREFIX + productId;

    // 1. 查询缓存
    String cacheData = redisService.get(cacheKey);

    if (cacheData == null) {
```

```
// 缓存不存在（可能是第一次查询），使用互斥锁重建
return rebuildCacheWithLock(productId);
}

// 2. 反序列化，检查逻辑过期时间
CacheData cache = JSON.parseObject(cacheData, CacheData.class);

if (cache.getExpireTime() < System.currentTimeMillis()) {
    // 3. 逻辑过期，异步更新缓存
    CompletableFuture.runAsync(() -> {
        rebuildCache(productId);
    });

    // 4. 返回旧数据（保证可用性）
    return cache.getData();
}

// 5. 未过期，直接返回
return cache.getData();
}
```

互斥锁 vs 逻辑过期：

维度	互斥锁	逻辑过期
性能	其他线程阻塞等待	无阻塞，立即返回旧数据
数据一致性	强一致	短暂数据不一致
实现复杂度	简单	复杂（需要异步更新）
适用场景	数据一致性要求高	允许短暂数据延迟

面试话术：

对于缓存击穿，我在项目中采用了**分布式锁 + 双重检查**方案：

- 使用SETNX实现分布式锁，保证只有一个线程重建缓存
- 其他线程等待50ms后再次检查缓存（双重检查）
- 如果缓存已重建，直接返回；否则降级查数据库

如果对性能要求更高，可以考虑**逻辑过期**方案：

- 缓存不设置TTL，而是存储逻辑过期时间
- 过期后返回旧数据，同时异步更新缓存
- 优点是无阻塞，缺点是短暂数据不一致

● 问题三：缓存雪崩（Cache Avalanche）

什么是缓存雪崩？

定义：大量缓存同时失效，请求全部打到数据库

场景1：系统重启

- 应用启动时批量刷新缓存
- 所有缓存设置相同的TTL（如30分钟）
- 30分钟后，1000个商品缓存同时过期

场景2：缓存服务器宕机

- Redis主节点宕机
- 从节点升为主节点，但缓存数据丢失

结果：

- 数据库CPU 100%
- 大量请求超时
- 数据库可能被打挂
- 级联故障，整个系统不可用

解决方案对比

方案	原理	优点	缺点
TTL随机化	基础TTL + 随机值	简单有效	不能完全避免同时过期
缓存预热	启动时预加载热点数据	避免冷启动	需要识别热点数据
多级缓存	本地缓存 + Redis	降低Redis压力	数据一致性问题
Redis高可用	哨兵/集群	避免单点故障	架构复杂度高

✅ 项目实现：TTL随机化 + 缓存预热

方案1：TTL随机化

核心代码：[ProductCacheServiceImpl.java:110-113](#)

```
/**
 * 将商品详情存入缓存
 * 【缓存雪崩防护】
 * 使用随机过期时间，避免大量缓存同时失效
 */
@Override
public void setProductToCache(ProductVO productVO) {
```

```

if (productVO == null || productVO.getId() == null) {
    return;
}

String cacheKey = PRODUCT_CACHE_PREFIX + productVO.getId();
Map<String, Object> cacheMap = productVOMap(productVO);

// 【关键】随机过期时间：基础时间 + 0~10分钟随机值
long randomTTL = CACHE_TTL_MINUTES + new Random().nextInt(10);
//          ↑ 基础30分钟      ↑ 随机0-10分钟
//          结果：30-40分钟范围内的随机值

redisService.hSetAll(cacheKey, cacheMap, randomTTL, TimeUnit.MINUTES);
}

```

效果对比：

无优化场景：

1000个商品，TTL都设置为30分钟

10:00:00 - 批量写入缓存

10:30:00 - 1000个缓存同时过期！

10:30:01 - 1000个请求同时打到DB

TTL随机化后：

1000个商品，TTL设置为30-40分钟随机值

10:00:00 - 批量写入缓存

10:30:00 - 约100个缓存过期（10%）

10:31:00 - 约100个缓存过期

10:32:00 - 约100个缓存过期

...

10:40:00 - 剩余缓存过期

结果：DB压力分散到10分钟内，QPS从1000降到100

方案2：缓存预热

核心代码：RedisCacheWarmer.java:44-73

```

/**
 * Redis 缓存预热
 * 【功能说明】
 * 应用启动时预加载热点数据到 Redis
 */
@Component
public class RedisCacheWarmer implements CommandLineRunner {
    //          ↑ 实现此接口，应用启动后自动执行run()

    @Autowired
    private StockCacheService stockCacheService;
}

```

```

/**
 * 应用启动后执行
 * 【实现说明】
 * 使用 CommandLineRunner 接口，在 Spring 容器初始化完成后执行
 */
@Override
public void run(String... args) {
    logger.info("===== 开始 Redis 缓存预热 =====");

    try {
        // 预热商品库存
        warmUpStock();

        logger.info("===== Redis 缓存预热完成 =====");
    } catch (Exception e) {
        // 【关键】预热失败不应影响应用启动
        logger.error("Redis 缓存预热失败，将使用懒加载模式", e);
    }
}

/**
 * 预热商品库存
 * 【作用】
 * 将所有商品库存加载到 Redis
 * 后续的库存扣减操作可以直接在 Redis 中进行
 */
private void warmUpStock() {
    logger.info("预热商品库存...");
    try {
        stockCacheService.syncAllStock(); // 同步所有库存到Redis
        logger.info("商品库存预热完成");
    } catch (Exception e) {
        logger.warn("商品库存预热失败：{}", e.getMessage());
    }
}
}

```

缓存预热的优势：

无预热场景：

1. 应用启动，Redis为空
2. 第一个用户访问商品 → 缓存miss → 查DB（100ms）
3. 用户体验差（首次访问慢）

有预热场景：

1. 应用启动 → 自动预加载热点数据到Redis
2. 用户访问 → 缓存hit → 返回数据（1ms）
3. 用户体验好（首次访问也快）

架构：

多级缓存架构

请求 → 本地缓存（Caffeine）

↓ miss

Redis缓存

↓ miss

MySQL数据库

本地缓存：

- 容量：1000条
- TTL：1分钟
- 优势：无网络开销，极快
- 劣势：数据不一致（多实例场景）

Redis缓存：

- 容量：100万条
- TTL：30分钟
- 优势：分布式共享
- 劣势：有网络开销

代码实现：

```
@Configuration
public class CacheConfig {

    @Bean
    public CacheManager cacheManager() {
        CaffeineCacheManager cacheManager = new CaffeineCacheManager();
        cacheManager.setCaffeine(Caffeine.newBuilder()
            .maximumSize(1000)
            .expireAfterWrite(1, TimeUnit.MINUTES));
        return cacheManager;
    }
}

// 使用
@Cacheable(value = "product", key = "#id")
public ProductVO getProduct(String id) {
    // 先查本地缓存，miss后再查Redis
    return productRepository.findById(id);
}
```

数据一致性：

问题：多个实例的本地缓存不一致

解决方案：

1. 短TTL：设置较短的过期时间（1分钟）
2. 主动失效：使用Redis Pub/Sub通知其他实例
3. 版本号：缓存数据携带版本号，版本不一致则重新加载

1.3 缓存更新策略（Cache Aside Pattern）

什么是Cache Aside（旁路缓存）？

旁路缓存：应用程序直接操作缓存和数据库，缓存和数据库是独立的

Cache Aside模式：

- 读取：先读缓存 → miss则读DB → 写缓存
- 更新：先更新DB → 再删除缓存

为什么要删除而不是更新缓存？

- 避免并发更新导致的数据不一致
- 删除成本低于更新成本（特别是复杂对象）

项目实现

缓存读取： [ProductCacheServiceImpl.java:138-154](#)

```
/**
 * 获取商品详情（优先缓存）
 * 【完整的缓存策略实现】
 * 1. 缓存穿透防护：缓存空值
 * 2. 缓存击穿防护：互斥锁重建
 * 3. 缓存雪崩防护：随机过期时间
 */
@Override
public ProductVO getProductWithCache(String productId) {
    String cacheKey = PRODUCT_CACHE_PREFIX + productId;

    // 1. 尝试从缓存获取（Cache Aside - Read）
    if (redisService.hasKey(cacheKey)) {
        Map<Object, Object> cacheMap = redisService.hGetAll(cacheKey);

        // 检查空值标记（缓存穿透防护）
        if (cacheMap != null && NULL_CACHE_VALUE.equals(cacheMap.get("_null"))) {
```

```

        return null;
    }

    if (cacheMap != null && !cacheMap.isEmpty()) {
        return mapToProductVO(cacheMap);
    }
}

// 2. 缓存未命中，从数据库加载 (Cache Aside - Read Through)
// ... 使用互斥锁防止缓存击穿
return loadAndCacheProduct(productId);
}

```

缓存删除: [ProductCacheServiceImpl.java:122-127](#)

```

/**
 * 删除商品缓存
 * 【Cache Aside Pattern】
 * 更新数据库后，删除缓存而不是更新缓存
 * 原因：避免并发更新导致的数据不一致
 */
@Override
public void deleteProductCache(String productId) {
    String cacheKey = PRODUCT_CACHE_PREFIX + productId;
    redisService.delete(cacheKey);
    //          ↑ 删除缓存，而非更新
}

```

业务层使用: [ProductController.java:96-105](#)

```

@PutMapping()
public Response<String> updateProduct(@RequestBody ProductVO productVO) {
    try {
        Product product = productVO.toProductPO();

        // 1. 【先更新数据库】
        productService.updateProduct(product);

        // 2. 【再删除缓存】 (Cache Aside - Write)
        if (productVO.getId() != null) {
            productCacheService.deleteProductCache(productVO.getId().toString());
        }

        return Response.buildSuccess("更新成功");
    } catch (TomatoMallException e) {
        return Response.buildFailure(e.getMessage());
    }
}

```

为什么先更新DB，再删除缓存？

错误示范：先删除缓存，再更新DB

线程A：删除缓存 → 更新DB（耗时100ms）

线程B：读缓存miss → 读DB（旧数据）→ 写缓存（旧数据）

线程A：更新完成

结果：缓存中是旧数据，DB是新数据（不一致）

正确做法：先更新DB，再删除缓存

线程A：更新DB → 删除缓存

线程B：读缓存miss → 读DB（新数据）→ 写缓存（新数据）

结果：最终一致

面试话术

我的商品详情缓存采用**Cache Aside**旁路缓存模式：

- **读取**：先查缓存，miss则查DB再写缓存
- **更新**：先更新DB，再删除缓存（而非更新缓存）
- **原因**：避免并发更新导致缓存和DB不一致

同时结合了三大缓存问题的解决方案：

- **缓存穿透**：缓存空值标记，TTL=5分钟
- **缓存击穿**：SETNX分布式锁 + 双重检查
- **缓存雪崩**：TTL随机化（30-40分钟）+ 缓存预热

2. 库存防超卖机制

2.1 什么是超卖？

场景：秒杀活动，商品库存只有1个

时间线：

T0：用户A查询库存 = 1

T1：用户B查询库存 = 1

T2：用户A扣减库存（库存变为0）

T3: 用户B扣减库存（库存变为-1）❌ 超卖！

结果：1个商品卖给了2个用户

2.2 传统方案的问题

方案1：MySQL乐观锁

```
// ❌ 有并发问题
Stockpile stock = stockRepository.findById(productId);
if (stock.getAmount() >= quantity) {
    stock.setAmount(stock.getAmount() - quantity);
    stockRepository.save(stock);
}
// 问题：读取和更新之间，其他线程可能修改了库存
```

方案2：MySQL悲观锁

```
// ⚠️ 性能差
@Transactional
public void decrStock(Integer productId, Integer quantity) {
    // SELECT ... FOR UPDATE（行锁）
    Stockpile stock = stockRepository.findByIdWithLock(productId);
    if (stock.getAmount() >= quantity) {
        stock.setAmount(stock.getAmount() - quantity);
        stockRepository.save(stock);
    }
}
// 问题：高并发下锁等待严重，QPS仅500左右
```

2.3 Redis原子操作方案

原理

Redis单线程模型：

- 所有命令串行执行
- DECR命令在服务端原子完成"读取-计算-写入"
- 不会出现竞态条件

DECRBY命令：

DECRBY key decrement

1. 将key的值减去decrement

2. 如果key不存在，先初始化为0再递减
3. 返回递减后的值

项目实施

核心代码：OrderServiceImpl.java:82-126

```
@Override
@Transactional
public OrderVO createOrder(String userName, CheckoutVO checkoutVO) {
    // ===== 【Redis 高并发防超卖方案】 =====

    // 2. 【Redis 原子扣减库存】 - 防超卖的核心
    // 使用 List 记录已扣减的商品，用于失败时回滚
    List<Integer> deductedProductIds = new ArrayList<>();
    List<Integer> deductedQuantities = new ArrayList<>();
    BigDecimal totalAmount = BigDecimal.ZERO;

    try {
        for (CartItem cartItem : cartItemList) {
            Integer productId = cartItem.getProductId();
            Integer quantity = cartItem.getQuantity();

            // 【关键步骤】调用 Redis DECR 原子扣减库存
            // decrStock 内部执行：DECRBY stock:product:{id} {quantity}
            // 返回值：扣减后的库存数量，如果 < 0 说明库存不足
            Long newStock = stockCacheService.decrStock(productId, quantity);
            //                               ↑ [StockCacheServiceImpl.java:92]

            if (newStock < 0) {
                // 库存不足，需要回滚之前所有已扣减的 Redis 库存
                rollbackRedisStock(deductedProductIds, deductedQuantities);
                throw TomatoMallException.stockpileNotEnough();
            }

            // 记录已扣减的商品（用于后续可能的回滚）
            deductedProductIds.add(productId);
            deductedQuantities.add(quantity);

            // 计算订单总金额
            Product product = productRepository.findById(productId).get();
            totalAmount = totalAmount.add(
                product.getPrice().multiply(new BigDecimal(quantity))
            );
        }

        // Redis 库存扣减全部成功，继续处理 MySQL 订单逻辑

    } catch (Exception e) {
        // 任何异常都需要回滚 Redis 库存
        rollbackRedisStock(deductedProductIds, deductedQuantities);
        throw e;
    }
}
```

```
// ... 后续订单创建逻辑
}
```

Redis扣减实现: [StockCacheServiceImpl.java:92-111](#)

```
/**
 * 扣减库存（原子操作）
 * 【核心实现】
 * 使用 DECRBY 原子递减库存
 */
@Override
public Long decrStock(Integer productId, Integer quantity) {
    String cacheKey = STOCK_CACHE_PREFIX + productId;
    //          ↑ stock:product:{productId}

    // 检查缓存是否存在，不存在则从数据库同步
    if (!redisService.hasKey(cacheKey)) {
        syncStock(productId);
    }

    // 【关键】原子递减
    Long newStock = redisService.decr(cacheKey, quantity);
    //          ↑ 底层执行: DECRBY stock:product:51 1

    // 如果库存为负，说明超卖，需要回滚
    if (newStock < 0) {
        // 回滚库存
        redisService.incr(cacheKey, quantity);
        //          ↑ 底层执行: INCRBY stock:product:51 1
        return -1L; // 返回 -1 表示库存不足
    }

    return newStock;
}
```

库存回滚: [OrderServiceImpl.java:207-214](#)

```
/**
 * 【Redis 库存回滚】
 * 当下单流程中任何步骤失败时，需要回滚已扣减的 Redis 库存
 */
private void rollbackRedisStock(List<Integer> productIds, List<Integer> quantities)
{
    for (int i = 0; i < productIds.size(); i++) {
        Integer productId = productIds.get(i);
        Integer quantity = quantities.get(i);
        // 调用 incrStock 回滚库存（内部执行 INCRBY）
        stockCacheService.incrStock(productId, quantity);
        //          ↑ 原子操作，使用INCRBY命令
    }
}
```

```
}  
}
```

性能对比

方案	QPS	实现复杂度	适用场景
MySQL悲观锁	~500	简单	低并发场景
MySQL乐观锁	~1000	中等	中等并发
Redis DECR	~100,000	简单	高并发场景

2.4 完整下单流程

高并发下单流程设计

- 1. 用户下单
↓
- 2. 【Redis DECR】原子扣减库存 ← 防超卖核心
 - └ 成功(newStock ≥ 0) → 继续流程
 - └ 失败(newStock < 0) → INCR 回滚，返回库存不足
↓
- 3. 【MySQL】创建订单记录（订单状态=待支付）
↓
- 4. 【延迟队列】30分钟后检查订单状态
 - └ 已支付 → 同步扣减 MySQL 库存
 - └ 未支付 → Redis INCR 恢复库存，取消订单

面试话术

我的库存管理采用 "Redis为主，MySQL为从" 的架构：

下单阶段（高并发）：

- 使用Redis DECR原子扣减库存
- Redis单线程模型保证无并发问题
- QPS可达10万+，远超MySQL的500 QPS
- 扣减失败则原子回滚（INCR操作）

支付阶段（低并发）：

- 支付成功后，同步扣减MySQL库存
- 最终一致性，MySQL作为持久化备份

超时处理：

- 订单加入延迟队列（ZSet实现）
- 30分钟后仍未支付，Redis恢复库存

代码位置：

- 扣减逻辑：[OrderServiceImpl.java:82](#)
- Redis操作：[StockCacheServiceImpl.java:92](#)

3. 分布式一致性方案

3.1 一致性问题分析

问题：Redis和MySQL如何保持一致？

场景：

1. 下单时：Redis扣减库存（真实库存）
2. 支付前：用户取消订单 → Redis恢复库存
3. 支付成功：MySQL扣减库存（同步Redis的值）

核心矛盾：

- Redis是实时库存（防超卖）
- MySQL是持久化备份（数据分析、BI报表）
- 需要保证最终一致

3.2 项目实施

支付成功同步MySQL

核心代码：[OrderServiceImpl.java:325-363](#)

```
/**
 * 【支付成功 - 同步 MySQL 库存】
 * 用户支付成功后，将 Redis 已扣减的库存同步到 MySQL
 *
 * 【执行时机】
```

```

    * updateOrderStatus 方法中，订单状态变更为 SUCCESS 时调用
    */
@Override
@Transactional
public void reduceStock(String orderId) {
    List<CartsOrdersRelation> cartsOrdersRelationList =
    cartsOrdersRelationRepository
        .findByOrderId(Integer.parseInt(orderId));

    for (CartsOrdersRelation cartsOrdersRelation : cartsOrdersRelationList) {
        CartItem cartItem =
    cartItemRepository.findById(cartsOrdersRelation.getCartItemId()).orElse(null);
        Product product =
    productRepository.findById(cartItem.getProductId()).orElse(null);

        Integer productId = product.getId();
        Integer quantity = cartItem.getQuantity();

        // 获取商品库存
        Stockpile stockpile = stockpileRepository.findByProductId(productId);

        // 【MySQL 同步】扣减实际库存
        // 注意：这里不再校验 amount 是否充足，因为：
        // 1. Redis 已在下单时原子扣减，保证不超卖
        // 2. 如果 MySQL 数据异常，以 Redis 为准
        if (stockpile.getAmount() >= quantity) {
            stockpile.setAmount(stockpile.getAmount() - quantity);
        } else {
            // MySQL 库存异常（小于应扣减量），直接设为 0 并记录日志
            System.err.println("警告：商品 " + productId + " MySQL库存异常，当前库存="
                + stockpile.getAmount() + "，应扣减=" + quantity);
            stockpile.setAmount(0);
        }

        // 清理 frozen 字段（不再使用）
        stockpile.setFrozen(0);
        stockpileRepository.save(stockpile);

        // 删除关系并从购物车删除条目
        cartsOrdersRelationRepository.delete(cartsOrdersRelation);
        cartItemRepository.deleteById(cartItem.getCartItemId());

        // 【Redis】从购物车缓存中删除
        cartCacheService.removeFromCart(cartItem.getUserId(),
    cartItem.getProductId());
    }
}

```

订单超时回滚

核心代码：[OrderServiceImpl.java:227-249](#)

```

/**
 * 释放订单锁定的库存（订单超时时调用）
 * 【重要】订单超时需要回滚 Redis 库存
 */
@Transactional
public void releaseLockedStock(String orderId) {
    List<CartsOrdersRelation> relations =
    cartsOrdersRelationRepository.findByOrderId(Integer.parseInt(orderId));

    for (CartsOrdersRelation relation : relations) {
        CartItem cartItem =
        cartItemRepository.findById(relation.getCartItemId()).orElse(null);
        if (cartItem == null) continue;

        Integer productId = cartItem.getProductId();
        Integer quantity = cartItem.getQuantity();

        // 【关键】回滚 Redis 库存（因为订单超时未支付）
        stockCacheService.incrStock(productId, quantity);
        //          ↑ 原子操作，使用INCRBY命令
        System.out.println("订单超时回滚：商品 " + productId + " 恢复库存 " +
        quantity);

        // 清理 MySQL frozen 字段（历史遗留，已不使用）
        Stockpile stockpile = stockpileRepository.findById(productId);
        if (stockpile != null) {
            int newFrozen = stockpile.getFrozen() - quantity;
            if (newFrozen < 0) newFrozen = 0;
            stockpile.setFrozen(newFrozen);
            stockpileRepository.save(stockpile);
        }
    }
}

```

延迟队列定时任务

核心代码： [OrderTimeoutTask.java:72-94](#)

```

/**
 * 处理超时订单
 * 【定时规则】
 * 每 10 秒执行一次
 */
@Scheduled(fixedRate = 10000) // 每10秒执行一次
public void processTimeoutOrders() {
    try {
        // 1. 获取到期的订单任务（最多100个）
        List<String> expiredTasks = delayQueueService.pollExpiredTasks(
            DelayQueueService.ORDER_TIMEOUT_QUEUE,
            100
        );
    }
}

```

```

        if (expiredTasks.isEmpty()) {
            return;
        }

        logger.info("发现 {} 个超时订单任务", expiredTasks.size());

        // 2. 逐个处理
        for (String taskData : expiredTasks) {
            processTimeoutOrder(taskData);
        }

    } catch (Exception e) {
        logger.error("处理超时订单异常", e);
    }
}

/**
 * 处理单个超时订单
 */
private void processTimeoutOrder(String taskData) {
    // 解析订单ID
    if (!taskData.startsWith("order:")) {
        logger.warn("无效的任务数据: {}", taskData);
        return;
    }

    String orderIdStr = taskData.substring(6); // 去掉 "order:" 前缀
    Integer orderId = Integer.parseInt(orderIdStr);

    // 查询订单
    Order order = orderRepository.findById(orderId).orElse(null);
    if (order == null) {
        logger.warn("订单不存在: {}", orderId);
        return;
    }

    // 检查订单状态
    if (order.getStatus() == Order.OrderStatus.PENDING) {
        // 订单仍为待支付状态，执行取消
        logger.info("取消超时订单: {}", orderId);

        // 更新订单状态为超时
        order.setStatus(Order.OrderStatus.TIMEOUT);
        orderRepository.save(order);

        // 释放锁定的库存
        orderService.releaseLockedStock(orderIdStr);

        logger.info("订单 {} 已超时取消，库存已释放", orderId);
    } else {
        // 订单已被处理（支付成功或已取消）
        logger.info("订单 {} 状态为 {}, 无需处理", orderId, order.getStatus());
    }
}

```

3.3 延迟队列实现（ZSet）

原理：

Redis ZSet作为延迟队列：

- Member: 订单ID（如 "order:123"）
- Score: 执行时间戳（当前时间 + 30分钟）

添加任务：

ZADD delay:queue:order:timeout <当前时间+30分钟> "order:123"

获取到期任务：

ZRANGEBYSCORE delay:queue:order:timeout 0 <当前时间>

删除已处理的任务：

ZREM delay:queue:order:timeout "order:123"

项目实施：[DelayQueueServiceImpl.java](#)

```
@Override
public void addOrderTimeoutTask(Integer orderId, int timeoutMinutes) {
    long executeTime = System.currentTimeMillis() + timeoutMinutes * 60 * 1000L;
    redisService.zAdd(DelayQueueKey, "order:" + orderId, executeTime);
    //          ↑ ZSet key    ↑ Member    ↑ Score (执行时间戳)
}

@Override
public List<String> pollExpiredTasks(String key, int count) {
    long currentTime = System.currentTimeMillis();

    // 获取分数在 0 到 当前时间 之间的元素
    Set<Object> tasks = redisService.zRangeByScore(key, 0, currentTime);

    List<String> result = new ArrayList<>();
    for (Object task : tasks) {
        if (result.size() >= count) break;

        String taskData = task.toString();

        // 原子删除（避免重复处理）
        if (redisService.zRemove(key, taskData) > 0) {
            result.add(taskData);
        }
    }

    return result;
}
```

3.4 数据一致性保障

数据一致性保障

1. Redis 为主（实时库存，防超卖）
 - 下单时：只扣 Redis
 - 查询库存：查询 Redis
2. MySQL 为从（持久化备份）
 - 支付成功：同步扣 MySQL
 - 对账系统：定时校验 Redis 和 MySQL 数据
3. 补偿机制
 - 订单超时：Redis 恢复库存
 - 支付失败：Redis 恢复库存 + 取消订单
4. 最终一致性
 - 允许短时间不一致（支付前）
 - 支付成功后保证一致

面试话术

我的方案不叫"双写一致性"，而是 **"分布式库存管理 + 最终一致性"**：

架构定位：

- Redis是库存中心（Source of Truth）
- MySQL是持久化备份
- 允许支付前短时间不一致

一致性保证：

- 支付成功后同步MySQL（事务保证）
- 订单超时回滚Redis库存
- 可引入对账系统定时校验

与Cache Aside的区别：

- Cache Aside：缓存是DB的副本
- 我的方案：Redis是独立的库存中心

如果MySQL扣减失败：

- 记录异常日志
- 以Redis为准
- 对账系统修正

4. JWT + Spring Security认证方案

4.1 当前实现分析

架构

当前实现（非标准）：

请求 → LoginInterceptor（拦截器）→ 验证Token → 存入Session → Controller

问题：

- 使用了HttpSession（有状态）
- 没有利用Spring Security的Filter链
- 没有标准的认证/授权流程

JWT生成

核心代码：TokenUtil.java:38-44

```
private static final long EXPIRE_TIME = 24 * 60 * 60 * 1000; // 24小时

public String getToken(Account account) {
    Date date = new Date(System.currentTimeMillis() + EXPIRE_TIME);
    return JWT.create()
        .withAudience(String.valueOf(account.getId())) // Payload: userId
        .withExpiresAt(date) // Payload: 过期时间
        .sign(Algorithm.HMAC256(account.getPassword())); // 签名密钥
    // ↑ 使用用户密码作为签名密钥
}

public boolean verifyToken(String token) {
    try {
        // 1. 解析Token获取userId
        Integer accountID =
            Integer.parseInt(JWT.decode(token).getAudience().get(0));

        // 2. 从数据库查询用户（获取最新密码作为签名密钥）
        Account account = accountRepository.findById(accountID).get();

        // 3. 使用最新密码验证签名
        JWTVerifier jwtVerifier =
```

```

JWT.require(Algorithm.HMAC256(account.getPassword())).build();
    jwtVerifier.verify(token);

    return true;
} catch (Exception e) {
    return false;
}
}

```

拦截器验证

核心代码: [LoginInterceptor.java:29-59](#)

```

@Override
public boolean preHandle(HttpServletRequest request, HttpServletResponse response,
Object handler) {

    // 白名单：注册接口不需要鉴权
    String uri = request.getRequestURI();
    String method = request.getMethod();
    if ("/api/accounts".equals(uri) && "POST".equalsIgnoreCase(method)) {
        return true;
    }

    // 获取请求头中的token
    String token = request.getHeader("token");

    if (token != null && tokenUtil.verifyToken(token)) {
        // Token验证成功，将用户信息存入Session
        request.getSession().setAttribute("currentAccount",
tokenUtil.getAccount(token));
        // ↑ 问题：使用了Session（有状态）
        request.setAttribute("accountId", tokenUtil.getAccount(token).getId());
        return true;
    } else {
        throw TomatoMallException.notLogin();
    }
}
}

```

4.2 标准实现方案

架构设计

Spring Security + JWT 标准架构

请求 → JwtAuthenticationFilter (Filter) → 验证Token → 存入SecurityContext → Controller

↓
完全无状态，不依赖Session

JWT工具类（优化版）

```
@Component
public class JwtTokenUtil {

    private static final long EXPIRE_TIME = 24 * 60 * 60 * 1000;
    private static final String SECRET_KEY = "your-secret-key"; // 统一密钥

    /**
     * 生成Token
     */
    public String generateToken(Integer userId, String username) {
        Date date = new Date(System.currentTimeMillis() + EXPIRE_TIME);
        return JWT.create()
            .withSubject(String.valueOf(userId)) // subject: 用户ID
            .withClaim("username", username) // 自定义声明: 用户名
            .withExpiresAt(date) // 过期时间
            .sign(Algorithm.HMAC256(SECRET_KEY)); // 统一密钥
    }

    /**
     * 验证Token
     */
    public boolean validateToken(String token) {
        try {
            JWTVerifier verifier =
                JWT.require(Algorithm.HMAC256(SECRET_KEY)).build();
            verifier.verify(token);
            return true;
        } catch (Exception e) {
            return false;
        }
    }

    /**
     * 从Token获取用户ID
     */
    public Integer getUserIdFromToken(String token) {
        return Integer.parseInt(JWT.decode(token).getSubject());
    }
}
```

JWT认证过滤器（核心）

```

@Component
public class JwtAuthenticationFilter extends OncePerRequestFilter {
    //          ↑ 确保一次请求只过滤一次

    @Autowired
    private JwtTokenUtil jwtTokenUtil;

    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
                                    FilterChain filterChain) throws
ServletException, IOException {

        // 1. 从Header获取token
        String token = request.getHeader("Authorization");

        if (token != null && token.startsWith("Bearer ")) {
            token = token.substring(7); // 去掉 "Bearer " 前缀

            try {
                // 2. 验证token
                if (jwtTokenUtil.validateToken(token)) {
                    // 3. 解析用户信息
                    Integer userId = jwtTokenUtil.getUserIdFromToken(token);

                    // 4. 创建认证对象
                    UsernamePasswordAuthenticationToken authentication =
                        new UsernamePasswordAuthenticationToken(
                            userId,                // principal (用户标识)
                            null,                  // credentials (密码)
                            Collections.emptyList() // authorities (权限列表)
                        );

                    // 5. 设置详细信息
                    authentication.setDetails(new
WebAuthenticationDetailsSource().buildDetails(request));

                    // 6. 将认证信息存入SecurityContext (关键!)

                    SecurityContextHolder.getContext().setAuthentication(authentication);
                    //          ↑ 无状态, 不依赖Session
                }
            } catch (Exception e) {
                // Token无效, 清除SecurityContext
                SecurityContextHolder.clearContext();
            }
        }

        // 7. 继续执行后续Filter
        filterChain.doFilter(request, response);
    }
}

```

```

@Configuration
@EnableWebSecurity
@EnableGlobalMethodSecurity(prePostEnabled = true) // 启用@PreAuthorize注解
public class SecurityConfig extends WebSecurityConfigurerAdapter {

    @Autowired
    private JwtAuthenticationFilter jwtAuthenticationFilter;

    @Autowired
    private JwtAuthenticationEntryPoint authenticationEntryPoint;

    @Autowired
    private JwtAccessDeniedHandler accessDeniedHandler;

    /**
     * 密码编码器
     */
    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    /**
     * 配置HTTP安全
     */
    @Override
    protected void configure(HttpSecurity http) throws Exception {
        http
            // 禁用CSRF（前后端分离项目必须禁用）
            .csrf().disable()

            // 禁用Session（完全无状态）
            .sessionManagement()
                .sessionCreationPolicy(SessionCreationPolicy.STATELESS)
            .and()

            // 配置授权规则
            .authorizeRequests()
                // 白名单：无需认证的接口
                .antMatchers("/api/accounts/login").permitAll()
                .antMatchers("/api/accounts/register").permitAll()
                .antMatchers("/api/orders/returnUrl").permitAll()
                .antMatchers("/api/orders/notify").permitAll()

                // 其他所有请求都需要认证
                .anyRequest().authenticated()
            .and()

            // 配置异常处理
            .exceptionHandling()
                .authenticationEntryPoint(authenticationEntryPoint) // 401
                .accessDeniedHandler(accessDeniedHandler) // 403
            .and()

            // 添加JWT过滤器（在UsernamePasswordAuthenticationFilter之前）
            .addFilterBefore(jwtAuthenticationFilter,

```

```
UsernamePasswordAuthenticationFilter.class);  
    }  
}
```

异常处理

401未认证:

```
@Component  
public class JwtAuthenticationEntryPoint implements AuthenticationEntryPoint {  
  
    @Override  
    public void commence(HttpServletRequest request,  
                        HttpServletResponse response,  
                        AuthenticationException authException) throws IOException {  
  
        response.setStatus(HttpServletResponse.SC_UNAUTHORIZED);  
        response.setContentType("application/json;charset=UTF-8");  
        response.getWriter().write("{\"code\":\"401\",\"message\":\"未登录或登录已过期  
\"}");  
    }  
}
```

403无权限:

```
@Component  
public class JwtAccessDeniedHandler implements AccessDeniedHandler {  
  
    @Override  
    public void handle(HttpServletRequest request,  
                    HttpServletResponse response,  
                    AccessDeniedException accessDeniedException) throws  
IOException {  
  
        response.setStatus(HttpServletResponse.SC_FORBIDDEN);  
        response.setContentType("application/json;charset=UTF-8");  
        response.getWriter().write("{\"code\":\"403\",\"message\":\"无权限访问\"}");  
    }  
}
```

获取当前用户

```
@RestController  
@RequestMapping("/api/products")  
public class ProductController {
```

```

/**
 * 从SecurityContext获取当前用户
 */
@GetMapping("/my-products")
public Response<List<Product>> getMyProducts() {
    // 获取当前用户的认证信息
    Authentication authentication =
SecurityContextHolder.getContext().getAuthentication();

    // 获取用户ID（我们在Filter中设置的是userId）
    Integer userId = (Integer) authentication.getPrincipal();

    // 查询用户的商品
    List<Product> products = productService.getByUserId(userId);

    return Response.buildSuccess(products);
}

/**
 * 使用注解控制权限
 */
@PreAuthorize("hasRole('ADMIN')") // 只有管理员可以访问
@DeleteMapping("/{id}")
public Response<String> deleteProduct(@PathVariable Integer id) {
    productService.deleteProduct(id);
    return Response.buildSuccess("删除成功");
}
}

```

4.3 对比分析

维度	当前实现	标准实现
无状态	✗ 使用Session	✓ 完全无状态
可扩展性	⚠ 需要Session共享	✓ 天然支持水平扩展
权限控制	✗ 手动实现	✓ @PreAuthorize注解
标准性	⚠ 自定义拦截器	✓ 符合Spring Security规范
异常处理	⚠ 手动抛异常	✓ 统一异常处理

面试话术

当前实现的问题：

1. 使用了 `HttpSession` 存储用户信息，是有状态的，不支持水平扩展

2. 没有利用Spring Security的Filter链，而是用自定义拦截器
3. 缺少细粒度的权限控制（角色/资源级别）

改进方案：

1. 用JWT Filter替代拦截器：继承 `OncePerRequestFilter`，在Spring Security Filter链中处理Token
2. 完全无状态：禁用Session，使用 `SecurityContextHolder`存储认证信息
3. 统一异常处理：实现 `AuthenticationEntryPoint`和 `AccessDeniedHandler`
4. 支持注解式权限控制：使用 `@PreAuthorize`注解

改造重点：

- 新增 `JwtAuthenticationFilter`替代 `LoginInterceptor`
- 重写 `SecurityConfig`，配置Filter链和授权规则
- 使用 `SecurityContextHolder.getContext().getAuthentication()`获取当前用户

5. 面试高频问题汇总

5.1 分布式事务与库存一致性

Q: 你提到了Redis和MySQL双写，如果支付成功了但MySQL扣库存失败怎么办？

参考回答：

在我的项目中，我采用了补偿机制保证最终一致性：

1. 当前实现：

- 支付成功后同步扣减MySQL库存
- 如果MySQL扣减失败，以Redis为准（Redis是库存真实来源）
- 通过定时任务对账，保证最终一致

2. 如果要优化（进阶回答）：

- 使用本地消息表（确保消息不丢失）
- 下单时发送"扣库存"消息到消息队列

- 消费者异步消费消息，扣减MySQL库存
- 如果失败，重试3次，仍失败则记录到异常表，人工处理

3. 其他方案：

- Seata AT模式：阿里巴巴开源的分布式事务框架
- Seata TCC模式：三阶段提交（Try-Confirm-Cancel）
- 本地消息表：最终一致性的轻量级方案

5.2 高并发下的库存超卖问题

Q: 如果Redis也挂了怎么办？你的方案还有什么兜底措施？

参考回答：

多层容错设计：

1. Redis高可用：

- Redis哨兵模式：主从复制 + 自动故障转移
- Redis集群：分片存储，降低单点风险

2. 降级方案：

- Redis不可用时，降级到MySQL悲观锁（虽然性能差，但保证不超卖）
- 使用熔断器（Hystrix/Resilience4j），快速失败

3. 预案设计：

- Redis预热：启动时从MySQL加载库存
- 限流：使用Guava RateLimiter或Redis+Lua实现令牌桶
- 库存预警：库存不足10个时，暂停该商品的秒杀活动

伪代码：

```
public Long decrStock(Integer productId, Integer quantity) {
    try {
        return redisService.decr(key, quantity);
    } catch (RedisConnectionException e) {
        // 降级到MySQL悲观锁
        return decrStockWithLock(productId, quantity);
    }
}
```

```
}  
}
```

5.3 订单超时处理的可靠性

Q: 你的延迟队列如果服务重启了，未处理的订单任务会不会丢失？

参考回答：

任务可靠性保证：

1. Redis持久化：

- 开启AOF持久化（Append Only File）
- 每秒刷盘（appendfsync everysec）
- 重启后从AOF恢复延迟队列数据

2. 任务重启恢复：

- 应用启动时，扫描MySQL中状态为PENDING的订单
- 检查订单创建时间，如果超过30分钟，立即取消
- 未超时的订单，重新加入延迟队列

3. 更优方案（进阶）：

- 使用RabbitMQ延迟队列（可靠性更高）
- 使用Redis + Redisson框架（内置延迟队列）
- 数据库轮询作为兜底（每分钟扫描一次）

5.4 JWT的安全问题

Q: 你的JWT如果被窃取了怎么办？有没有考虑过Token刷新机制？

参考回答：

Token安全问题：

1. 当前实现的局限：

- Token有效期24小时，被窃取后长时间有效

- 没有主动撤销Token的机制

2. 优化方案（进阶）：

- **短期Token + 长期RefreshToken：**
 - Access Token：15分钟有效期
 - Refresh Token：7天有效期，存储在Redis
 - Access Token过期后，用RefreshToken换取新的Access Token
- **Token黑名单：**
 - 用户登出时，将Token加入Redis黑名单
 - 拦截器验证时检查黑名单
 - 黑名单TTL与Token过期时间一致
- **IP绑定（可选）：**
 - Token生成时记录用户IP
 - 验证时检查IP是否一致

5.5 数据库设计与索引优化

Q: 你的订单表是怎么设计的？如果订单表有几千万条数据，查询会不会很慢？

参考回答：

订单表优化方案：

1. 索引设计：

```
-- 用户查询订单（高频）
CREATE INDEX idx_user_id_status ON orders(user_id, status, create_time DESC);

-- 商家查询订单
CREATE INDEX idx_seller_id ON orders(seller_id, create_time DESC);

-- 订单超时扫描
CREATE INDEX idx_status_create_time ON orders(status, create_time);
```

2. 分库分表（进阶）：

- 水平分表：按user_id取模，分64张表

- 垂直分库：订单主表、订单商品表、订单状态变更表
- 历史订单归档：3个月前的订单迁移到历史表

3. 冷热分离：

- 热数据（3个月内）：MySQL主库
- 温数据（3-12个月）：MySQL从库
- 冷数据（1年以上）：归档到对象存储或ClickHouse

5.6 支付回调的幂等性

Q: 如果支付宝重复回调了怎么办？ 你如何保证不会重复扣款？

参考回答：

支付回调幂等性保证：

1. 当前实现：

- 更新订单状态时检查当前状态
- 只有PENDING状态才能变更为SUCCESS

2. 优化方案：

- 分布式锁：

```
public void updateOrderStatus(String orderId) {
    String lockKey = "lock:order:pay:" + orderId;
    boolean locked = redis.setIfAbsent(lockKey, "1", 10, TimeUnit.SECONDS);

    if (!locked) {
        throw new RuntimeException("重复回调");
    }

    try {
        Order order = orderRepository.findById(orderId);
        if (order.getStatus() != OrderStatus.PENDING) {
            return; // 已处理，直接返回
        }
        order.setStatus(OrderStatus.SUCCESS);
        orderRepository.save(order);
    } finally {
        redis.delete(lockKey);
    }
}
```

- 状态机：

PENDING → SUCCESS
→ TIMEOUT
→ CANCELLED

状态只能向前推进，不能回退

5.7 项目难点与收获

Q: 你在项目中遇到的最大技术难点是什么？你是如何解决的？

参考回答：

难点1：库存扣减的并发问题

- 问题：高并发下MySQL乐观锁导致大量重试失败
- 解决：改用Redis原子操作，QPS从1000提升到10万

难点2：延迟队列的可靠性

- 问题：服务重启后延迟队列任务丢失
- 解决：增加启动时的任务恢复逻辑

难点3：支付回调的幂等性

- 问题：支付宝可能重复回调
- 解决：状态检查 + 分布式锁

5.8 系统设计题

Q: 如果要支持双11大促，你的系统需要做哪些优化？

参考回答：

1. 架构层面：

- 服务降级：关闭评论、推荐等非核心功能
- 限流：用户维度限流（每秒1个请求）

- 熔断：依赖服务故障时快速失败

2. 缓存层面：

- 预热：提前加载热点商品到Redis
- 多级缓存：本地缓存 + Redis
- 缓存更新：使用Canal监听MySQL binlog，实时更新Redis

3. 数据库层面：

- 读写分离：主库写，从库读
- 分库分表：按user_id分表
- 冷热分离：历史订单归档

4. 监控告警：

- Prometheus + Grafana监控
- 库存不足告警
- 接口响应时间告警



面试准备清单

重点准备的主题

主题	优先级	代码位置
Redis缓存三大问题	☆☆☆	ProductCacheServiceImpl.java
库存防超卖	☆☆☆	OrderServiceImpl.java:82
分布式一致性	☆☆☆	OrderServiceImpl.java:325
JWT认证	☆☆	TokenUtil.java
订单超时处理	☆☆	OrderTimeoutTask.java
Spring Security	☆☆	SecurityConfig.java

自我检查清单

- ☐ 能流畅解释缓存三大问题及解决方案

- ☐ 能画出库存扣减的完整流程图
 - ☐ 能说出Redis vs MySQL方案的性能差异
 - ☐ 能解释JWT的工作原理和优缺点
 - ☐ 能说明当前实现的问题和改进方向
 - ☐ 能回答分布式事务的常见方案
 - ☐ 能设计一个简单的秒杀系统架构
-

总结

项目亮点

1. Redis缓存体系完整：

- 使用5种数据结构（String/Hash/List/Set/ZSet）
- 解决缓存三大问题（穿透/击穿/雪崩）
- 支持PV/UV统计、热搜榜、延迟队列

2. 库存管理创新：

- Redis原子操作防超卖（10万QPS）
- 分布式一致性保证
- 订单超时自动回滚

3. 第三方集成：

- 阿里云OSS文件存储
- 支付宝沙箱支付
- 通义千问AI客服

4. 可优化的点：

- Spring Security标准改造
- 消息队列引入
- 分库分表扩展

面试策略

1. 先讲实现：详细讲解自己的代码实现

2. **再讲对比**：对比不同方案的优缺点

3. **最后讲改进**：说明可以优化的方向

核心话术：

"我的项目采用了Redis+MySQL的分布式库存管理架构，通过Redis原子操作实现高并发防超卖，通过延迟队列和补偿机制保证最终一致性。相比传统MySQL方案，QPS从500提升到10万，同时支持水平扩展。"

祝你面试顺利！🎉